

Beyond VCD: XTrace for AI-Native RTL Debug

A popular technical paper on AI-readable simulation traces for RTL verification agents

Bondan Rufen | bondanr@yahoo.com | X: @bondanr | XTrace GitHub: github.com/aionhw/xtrace

Abstract - VCD helped define decades of RTL simulation debug, but it was designed for waveform viewers, not AI agents. An AI debug agent needs compact, structured, semantic evidence: instruction events, transaction context, state changes, assertion failures, and focused failure windows. XTrace is proposed as an AI-readable trace format that complements VCD and can become the default simulator-to-agent interface for AI-native verification. In an early PicoRV32 experiment using testbench_ez.v, an intentional ADDI decode bug was inserted and Opus 4.7 was asked to debug from VCD versus XTrace after learning RV32IMAC, VCD, and XTrace. In this small test, XTrace was estimated to help the model converge about 2x faster. The result is directional, not final, but it shows why trace representation matters.

Keywords - XTrace, VCD, AI-native verification, RTL debug, PicoRV32, RISC-V, simulation traces

At a glance

Question	VCD	XTrace
Primary abstraction	Signal toggles over time	Semantic debug events and failure evidence
Agent effort	Map symbols, filter noise, infer meaning	Reason from explicit instruction/transaction context
Early PicoRV32 result	More tool calls and detours	~2x faster convergence in the ADDI decode experiment

1. Motivation: VCD Is Precise, but Not AI-Native

Value Change Dump (VCD) is durable because it is universal: simulators can emit it, waveform viewers can load it, and engineers know how to inspect it. It records signal transitions with enough fidelity to reconstruct behavior at the waveform level. For human waveform debug, that has been useful for decades.

The problem is that AI agents do not debug like waveform viewers. A language model must spend context and reasoning budget mapping short VCD identifiers back to signal names, following clock edges, ignoring reset and clock noise, reconstructing instruction or transaction meaning, and then inferring why a failure happened. VCD gives the model a detailed record of what toggled, but little help understanding what mattered.

2. What XTrace Changes

XTrace is a compact, structured trace format designed to communicate simulation behavior to AI agents. It is not merely a compressed VCD: compression reduces size, while XTrace changes representation. VCD answers "what toggled?" XTrace answers "what happened, where did it happen, and why might it matter?"

AI-native debug hierarchy

- Default input: XTrace debug packet with failure context and suspect window.
- More detail: XTrace signal slice around the failure event.
- Full fidelity fallback: VCD, FST, or FSDB for exact waveform inspection.

3. Experiment: PicoRV32 ADDI Decode Bug

The experiment used PicoRV32 with testbench_ez.v. The injected bug was intentionally simple: the decode logic for ADDI was modified so that ADDI instructions were misclassified or not recognized correctly. ADDI is a good first benchmark because it appears early in many RISC-V programs for register initialization, stack setup, immediate arithmetic, and loop increments.

```
Injected bug class
  expected: opcode/funct3 pattern activates ADDI decode
  actual:   ADDI is not recognized or is mapped to the wrong decode path
  effect:   missing or incorrect register writeback after an ADDI instruction
```

Opus 4.7 was first asked to learn the required context: RV32IMAC behavior, VCD, and XTrace. It was then given two separate debug inputs, one based on VCD and one based on XTrace. The comparison focused on characters, estimated tokens, tool-call rounds, detours, and time to identify the likely root cause.

```

... +41 lines (ctrl+o to expand)
Searched for 5 patterns, read 2 files (ctrl+o to expand)
Bug found
File: picorv32/picorv32.v:1057
instr_addi <= is_alu_reg_imm && mem_rdata_q[14:12] == 3'b001; // BUG_INJECT
ADDI is funct3 = 3'b000, not 3'b001. (3'b001 is SLLI.) The // BUG_INJECT comment confirms it's a planted bug. The neighbors (lines 1058-1062) follow the correct
sltiu=011, xori=100, ori=110, andi=111. The 000 slot was swapped to 001.
How the VCD evidence pinpointed it

```

Time	Signal	Value	Meaning
#1050	cpu_state (b)	0x20	entered ld_rs1
#1050	dbg_insn_addr (r)	0x00000000	first instruction at PC=0
#1050	dbg_insn_opcode (t)	0x3FC00093	addi x1, x0, 1020 (funct3=000, opcode=0010011)
#1050	instr_addi (i!)	0 -- wrong, should be 1	
#1050	instr_trap (^!)	1 -- decoder said "illegal"	
#1050	is_lui_avipc_jal_jalr_addi_add_sub (t!)	0 -- wrong, ADDI is in this group	
#1060	cpu_state (b)	0x80	cpu_state_trap
#1060	dbg_ascii_state (q)	"trap"	confirmed
#1070	trap (*)	1	trap output asserted

Figure 1. Opus 4.7 debug result using the PicoRV32 VCD trace. The model found the ADDI decode bug, but had to reconstruct the evidence from waveform-level records and cross-referenced signals.

4. Why XTrace Helps the Agent Reason Faster

With VCD, the model must reconstruct meaning from low-level transitions: resolve symbol aliases, track time steps, follow PC and instruction signals, infer that a bit pattern is OP-IMM ADDI, correlate expected writeback against actual behavior, and then suspect decode. This is bookkeeping before real reasoning begins.

With XTrace, the same failure can be expressed closer to design semantics. The trace can expose the instruction event directly and attach expected versus actual decode information, without hiding raw instruction bits or signal references.

```

event=instruction pc=0x00000080 raw=0x00100093 isa=RV32I op=ADDI
decode.expected=OP_IMM.ADDI decode.actual=none
rd=x1 rs1=x0 imm=1 writeback.expected=x1:0x00000001
writeback.actual=missing suspect=decode.instr_addi

```

```

Root cause found
picorv32.v:1057 has a planted bug (marked // BUG_ADDI):
instr_addi <= is_alu_reg_imm && mem_rdata_q[14:12] == 3'b001; // BUG_ADDI
ADDI's funct3 is 3'b000, not 3'b001. So the decoder never asserts instr_addi for any actual ADDI instruction.
Trace evidence (testbench_ez.xt)
- Line 595 - reset releases (i=1, j=1)
- Line 603 - first ifetch begins: mem_addr=0, mem_instr=1, mem_valid=1
- Line 607 - memory returns 0x3fc00093 -> addi x1, x0, 1020 (opcode 0010011, funct3 000, rd=x1, imm=0x3FC)
- Line 616-618 - decode result: count_instr=1, dbg_insn_opcode=0x3fc00093, dbg_ascii_instr=0 (empty - no match), sa=0x1 (instr_trap),
  none of J (instr_addi), 2 (instr_lui), W (instr_jal)... are set
- Line 626 - j=1 and Mb=1 (top-level trap and uut.trap latch high), CPU is dead from here on (it just keeps clocking, no further
  fetches)
Fix
Change line 1057 to:
instr_addi <= is_alu_reg_imm && mem_rdata_q[14:12] == 3'b000;
Want me to apply the fix?

```

Figure 2. Opus 4.7 debug result using the PicoRV32 XTrace. The XTrace path surfaced the key decode evidence directly, including the ADDI funct3 mismatch and the suggested fix.

5. Early Result and Interpretation

In this small PicoRV32 test, Opus 4.7 estimated that XTrace helped it converge on the injected ADDI decode bug roughly 2x faster than VCD. The important result is not the exact multiplier; it is the direction. When debug information is represented as structured evidence instead of raw waveform activity, the AI agent spends less effort reconstructing context and more effort localizing the bug.

	VCD	XTrace
Tool-call rounds to localize bug	~10	~5
Tool calls total	18	7
Files I had to cross-reference	VCD + testbench.v + golden.trace + picorv32.v	XTrace + picorv32.v
Detours (false starts / dead ends)	testbench.trace empty (had to look at VCD), trap signal at line 1438 needed timestamp lookup, signal IDs 1!,^!,l!,p!,t! had to be grep'd individually	none – packed P, records show all decoder flags in one line at #1050

Why XTrace was faster despite similar token count:

- One P,... record at #1050 surfaced instr_addi=0, instr_trap=1, is_lui..._addi_add_sub=0, dbg_insn_opcode=0x3fc00093 together. In VCD those were 5 separate 0x! deltas across 50 lines.
- Self-documenting IDs (J,sa,Ha,B) are at least typed/named in the dict; VCD's 1!,sa,^!,t! collide visually with value chars and require careful grepping.
- No need to consult testbench.v or golden.trace to confirm the trap was real – j=0x1 and Mb=0x1 (top-level + uut.trap) are both in the same record.

Rough estimate: XTrace path was ~2x faster end-to-end, even though token count was the same. The savings come from fewer round-trips, not fewer tokens. For an interactive debug session that matters a lot; for a single batched LLM call it matters less.

Figure 3. VCD vs. XTrace debug results on the PicoRV32 ADDI decode bug. XTrace reduced round-trips and detours even when token count was similar, because the relevant evidence was packed into fewer, more meaningful records.

Input	Character/token pressure	Agent burden	Observed direction
VCD	Higher for equivalent context	Map symbols, infer instruction meaning, filter noise	Slower convergence
XTrace signal slice	Lower and more readable	Follow named signals and local window	Faster localization
XTrace debug packet	Lowest for failure triage	Reason from semantic evidence	Fastest hypothesis path

Important caveat

- This is an early directional experiment, not a controlled benchmark.
- The design is small and the injected bug is localized.
- The next step is to repeat the test across bug classes, designs, and models.

6. Why the Advantage Should Grow on Larger Designs

The PicoRV32 experiment is intentionally modest, which makes the result useful. VCD is still manageable at this scale. In larger CPUs, accelerators, interconnects, or SoCs, the gap should widen because VCD grows with signal activity while XTrace can be designed to grow with debug relevance.

A complex design contains many clocks, resets, pipeline stages, queues, protocol states, and internal control signals. Most toggles are irrelevant to any specific failure. A human engineer handles this by filtering the waveform mentally and visually. An AI agent needs the simulator or trace generator to perform part of that filtering explicitly.

7. Benchmark and Deployment Path

The next step is to make the experiment reproducible. A stronger benchmark should measure characters, estimated tokens, time to first useful hypothesis, time to correct root cause, prompt count, false suspicion rate, and scaling across tests. Good bug classes include ADDI decode, branch decode, load/store alignment, writeback enable, pipeline hazard, bus handshake, reset sequencing, and multicore communication failures.

Flow stage	Traditional debug	AI-native XTrace debug
Simulation output	Full waveform dump first	XTrace packet first; waveform optional
Initial triage	Engineer opens waveform and filters manually	AI summarizes failure, suspect window, and candidates
Deep debug	Manual signal tracing across time	Agent requests focused slices or VCD windows
Regression loop	Repeat diagnosis for each failure	Cluster similar failures and track deltas

8. Design Principles for XTrace

A practical XTrace format should follow five principles: explicit naming, locality, reversibility, loss awareness, and determinism. Explicit names reduce the model cost of VCD-style symbol aliases. Locality keeps failure packets near the relevant reset, instruction, transaction, and state context. Reversibility lets an engineer jump back to original time ranges and signals. Loss awareness distinguishes lossy summaries from lossless signal slices. Determinism keeps debug reproducible.

9. Future Work

The immediate next step is to publish a reproducible benchmark package. That package should include the injected PicoRV32 bug, the failing testbench, the VCD input, the XTrace input, prompt templates, model logs, character counts, token estimates, and the final root-cause report. The benchmark should separate three measurements: raw trace size, model context cost, and end-to-end debug convergence.

Benchmark item	Purpose
Bug suite	ADDI decode, branch decode, load/store, writeback, reset, bus handshake, and pipeline hazard failures
Model set	Compare Opus, GPT, Gemini, and smaller local models using the same evidence packets
Trace modes	VCD, XTrace signal slice, XTrace transaction layer, and XTrace debug packet
Success metric	Correct root cause with fewer prompts, fewer tool calls, and lower false-suspicion rate

10. Availability

The XTrace repository is available at <https://github.com/aionhw/xtrace>. The intent is to make the trace format practical for simulator integration and easy for AI agents to parse. The format should remain simple enough for direct inspection by verification engineers, while still carrying enough structure for automated triage.

11. Conclusion

VCD remains valuable as a ground-truth waveform record, but it is not the best default interface for AI agents. It records what toggled. It does not naturally record what happened. As AI becomes part of the verification loop, simulators need to emit traces that are compact, semantic, hierarchical, and failure-focused. XTrace is an attempt to define that interface. The early PicoRV32 ADDI decode experiment suggests that even a small design can benefit, with an estimated 2x faster convergence using XTrace than VCD. For complex designs, the advantage should be larger because the cost of reconstructing meaning from raw waveform dumps grows with design scale.

VCD was built for waveform viewers. XTrace is built for reasoning agents.